

Runtime Struct Navigation in Compiled Languages Without Reflection or Code Generation

Comptime Struct Registry

Registry: [\[HOWL-COMP-13-2026\]](#)

Series Path: [\[HOWL-COMP-1-2026\]](#) → [\[HOWL-COMP-2-2026\]](#) → [\[HOWL-COMP-3-2026\]](#) → [\[HOWL-COMP-4-2026\]](#) → [\[HOWL-COMP-5-2026\]](#) → [\[HOWL-COMP-6-2026\]](#) → [\[HOWL-COMP-7-2026\]](#) → [\[HOWL-COMP-8-2026\]](#) → [\[HOWL-COMP-9-2026\]](#) → [\[HOWL-COMP-10-2026\]](#) → [\[HOWL-COMP-11-2026\]](#) → [\[HOWL-COMP-12-2026\]](#) → [\[HOWL-COMP-13-2026\]](#)

DOI: 10.5281/zenodo.20846381

Date: June 2026

Domain: Systems Programming / Build Infrastructure / Runtime Data Access

Status: Working Implementation

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet.

Abstract

Compiled languages discard struct layout information after compilation. Programs that need runtime access to field metadata — for serialization, property editors, data binding, or path-based navigation — resort to runtime reflection, code generation, or manual registration. All three scale poorly with struct count. This paper presents a fourth approach: capture struct field metadata at build time into a static descriptor table, then walk arbitrary nested structs at runtime through pointer arithmetic using pre-computed offsets. The technique requires no runtime type information, no generated source files, no dynamic dispatch, and no heap allocation. A working implementation covering 140+ structs reduced compile time from 15 seconds to 2 seconds by eliminating the code generation pipeline it replaced. The approach is portable to any compiled language with access to `offsetof` or equivalent.

1. The Problem

A compiled language knows everything about every struct at compile time. Field names, byte offsets from the struct base, byte sizes, types, nesting depth, slice element types. The compiler uses this information to generate correct machine code, then discards it. The resulting binary cannot answer basic questions: what fields does this struct have? What is the value at `player.inventory.items.3.weight`?

Three workarounds exist.

Runtime reflection is available in Java, C#, and Python. It is unavailable in C, limited in C++, and deliberately excluded from Zig. Languages that provide it pay for it with runtime overhead — type metadata tables, dynamic dispatch through vtables, heap allocation for type descriptors. Languages that exclude it do so for good reason: reflection conflicts with the zero-overhead principle that motivates choosing a systems language in the first place.

Code generation is the standard approach in systems languages. A tool reads struct definitions (from source files, header files, or a separate schema), then produces source files containing serializers, deserializers, registration functions, and editor builders — one set per struct. These generated files enter the normal compilation pipeline. The costs: the generator tool must be maintained, the generated output must stay synchronized with the source definitions, and the generated code inflates compile time proportionally to struct count. In the project that motivated this paper, 140+ structs produced thousands of generated lines, inflating compile time from 1 second to 15 seconds.

Manual registration requires the developer to write registration calls per field per struct at startup. Register the name, the offset, the type, the size. For 140 structs with 20 fields each, that is 2,800 registration calls maintained by hand. Any field addition, removal, or rename requires a corresponding change to the registration code. Drift between struct definitions and registration is guaranteed over time.

All three approaches share a structural flaw: they scale linearly with struct count. Every new struct added to the project increases the maintenance burden of whichever approach is in use.

2. The Technique

A struct field is bytes at an offset from a base pointer. Given the struct:

```
const Player = struct {  
  
    health: i32 = 100,  
  
    speed: f32 = 5.0,  
  
    name: Text = Text.init("unnamed"),  
  
};
```

The compiler determines that `health` is 4 bytes at offset 0, `speed` is 4 bytes at offset 4, and `name` is 4096+ bytes at offset 8 (exact values depend on alignment). These numbers fully describe how to access any field at runtime: take a pointer to a `Player` instance, add the offset, interpret the bytes according to the type.

The technique has three steps:

1. At build time, iterate every struct's fields and record each field's name, byte offset, byte size, and type classification into a static array.
2. For nested structs and slices, record the index of the child struct's descriptor in the same array, creating a navigable type graph.
3. At runtime, resolve a path string segment by segment: look up each field name in the current struct's descriptor, add the offset to the current pointer, follow child type indices for nested structs, index by element size for slices.

No runtime type information is involved. No dynamic dispatch. No heap allocation. The descriptor table is static data computed once at build time and embedded in the binary. The runtime cost is a name lookup in a flat array (at most 96 entries per struct) plus pointer addition per path segment.

3. Data Structures

The registry consists of three structs.

3.1 FieldDescriptor

```
pub const FieldDescriptor = struct {
    name: [64]u8 = [_]u8{0} ** 64,
    name_len: usize = 0,
    offset: usize = 0,
    size: usize = 0,
    field_type: FieldType = .unknown,
    child_type_id: usize = 0,
    slice_element_size: usize = 0,
};
```

Each field descriptor stores the field name in a fixed 64-byte buffer (no heap allocation, no pointer), the byte offset from the containing struct's base address, the byte size of the field, a type classification tag, and two fields for navigating into nested structures.

`child_type_id` is an index into the registry's descriptor array. When `field_type` is `.struct_val`, this index identifies which `StructDescriptor` describes the nested struct's fields. When `field_type` is `.slice`,

this index identifies the element type, and `slice_element_size` stores the byte size of each element for indexing.

3.2 StructDescriptor

```
pub const StructDescriptor = struct {  
    name: [64]u8 = [_]u8{0} ** 64,  
    name_len: usize = 0,  
    struct_size: usize = 0,  
    field_count: usize = 0,  
    fields: [96]FieldDescriptor = [_]FieldDescriptor{.{}} ** 96,  
};
```

Each struct descriptor stores the struct name, total byte size, field count, and up to 96 field descriptors. The fixed-size field array avoids any heap allocation. 96 fields per struct is sufficient for all structs encountered in the reference project including a 30-subsystem entity struct.

3.3 WalkResult

```
pub const WalkResult = struct {  
    ptr: *anyopaque,  
    field: FieldDescriptor,  
};
```

The return type of the path walker. Pairs a raw pointer to the field's memory location with the descriptor explaining how to interpret the bytes. The caller uses the `field_type` tag to select the appropriate cast: `@ptrCast(@alignCast(result.ptr))` to `*i32`, `*f32`, `*bool`, `*Text`, or any other concrete type.

4. Type Classification

```
pub const FieldType = enum {
```

```
int_i32,  
float_f32,  
bool_val,  
uint_u8,  
uint_u16,  
int_usize,  
text,  
string,  
enum_i32,  
struct_val,  
slice,  
vector2,  
unknown,  
};
```

A build-time function maps each compile-time type to this runtime tag:

```
fn classifyFieldType(comptime T: type) FieldType {  
    if (T == i32) return .int_i32;  
    if (T == f32) return .float_f32;  
    if (T == bool) return .bool_val;  
    if (T == u8) return .uint_u8;  
    if (T == u16) return .uint_u16;  
    if (T == usize) return .int_usize;  
    if (@typeInfo(T) == .@"enum") return .enum_i32;
```

```

if (@typeInfo(T) == .@"struct") {
    if (T == Text) return .text;
    if (T == String) return .string;
    if (@hasField(T, "x") and @hasField(T, "y") and !@hasField(T, "z")) {
        if (@FieldType(T, "x") == f32) return .vector2;
    }
    return .struct_val;
}

if (@typeInfo(T) == .pointer) {
    if (@typeInfo(T).pointer.size == .slice) return .slice;
}

return .unknown;
}

```

The function checks exact type identity for primitive types, then falls through to structural checks for compound types. Custom application types (Text, String) are detected by type identity. Vector2-like types are detected by field signature — has `x` and `y` fields of type `f32`, does not have `z`. All enums collapse to `enum_i32` because every enum in the reference project is backed by `i32`.

This enum is the bridge between compile time and runtime. After compilation, the original type is gone. The tag remains, telling runtime code how to interpret the bytes at a given offset.

5. Building the Registry

```

fn buildRegistry(comptime types: anytype) [types.len]StructDescriptor {
    @setEvalBranchQuota(1000000);

    var descriptors: [types.len]StructDescriptor = [_]StructDescriptor{.{}} **

```

```

inline for (types, 0..) |T, type_idx| {
    if (@TypeInfo(T) != @"struct") continue;
    var sd = StructDescriptor{};
    // Extract short name from fully qualified type name
    var type_name_full = @typeName(T);
    var last_dot: usize = 0;
    for (type_name_full, 0..) |c, i| {
        if (c == '.') last_dot = i + 1;
    }
    sd.name_len = copyName(&sd.name, type_name_full[last_dot..]);
    sd.struct_size = @sizeof(T);
    sd.field_count = @TypeInfo(T).@"struct".fields.len;
    inline for (@TypeInfo(T).@"struct".fields, 0..) |field, field_idx| {
        var fd = FieldDescriptor{};
        fd.name_len = copyName(&fd.name, field.name);
        fd.offset = @offsetof(T, field.name);
        fd.size = @sizeof(field.type);
        fd.field_type = classifyFieldType(field.type);
        if (fd.field_type == .struct_val) {
            fd.child_type_id = findTypeIndex(types, field.type);
        } else if (fd.field_type == .slice) {
            const ElemT = sliceElementType(field.type);
            fd.slice_element_size = @sizeof(ElemT);
        }
    }
}

```

```

        if (@typeInfo(ElemT) == .@"struct") {

            fd.child_type_id = findTypeIndex(types, ElemT);

        }

    }

    sd.fields[field_idx] = fd;

}

descriptors[type_idx] = sd;

}

return descriptors;

}

```

This function runs entirely at compile time. It receives a tuple of struct types, iterates each one, and for each field records the name, offset, size, and classified type. For struct-valued fields, `findTypeIndex` locates the child type's position in the tuple and stores it as `child_type_id`. For slice fields, the element type is extracted, its size stored as `slice_element_size`, and if the element is a struct, its type index stored as `child_type_id`.

The output is a fixed-size array of `StructDescriptor` that the compiler embeds as static data in the binary. No code is generated. No source files are produced. No build step runs. The compiler evaluates the function during compilation and the result exists as constant data.

Usage is a single declaration:

```

const ALL_TYPES = .{

    Entity,

    EntityTransform,

    ResourceInstance,

    ResourceDefault,

    IconStrip,

    MenuData,

```



```
// ... all structs
};

const registry = buildRegistry(ALL_TYPES);
```

Adding a struct to the system means adding one line to this tuple.

6. Runtime Path Walking

```
pub fn walkPath(
    segments: []const []const u8,
    root_ptr: [*]u8,
    root_type_id: usize,
) !WalkResult {
    var cur_ptr = root_ptr;
    var cur_type_id = root_type_id;
    var seg_idx: usize = 0;
    while (seg_idx < segments.len) {
        var segment = segments[seg_idx];
        var sd = registry[cur_type_id];
        // Check if segment is a numeric index (for slices)
        var maybe_index: ?usize = std.fmt.parseInt(usize, segment, 10) catch n
        if (maybe_index) |index| {
            var slice_ptr_val = @as(*const [*]u8, @ptrCast(@alignCast(cur_ptr)
            var element_size = registry[cur_type_id].struct_size;
            cur_ptr = slice_ptr_val + (index * element_size);
```

```

        seg_idx += 1;

        continue;
    }

    // Named field lookup
    var found = false;
    for (sd.activeFields()) |fd| {
        if (std.mem.eql(u8, fd.nameSlice(), segment)) {
            var field_ptr = cur_ptr + fd.offset;

            // Last segment - return result
            if (seg_idx == segments.len - 1) {
                return WalkResult{
                    .ptr = @ptrCast(@alignCast(field_ptr)),
                    .field = fd,
                };
            }

            // Navigate deeper
            if (fd.field_type == .struct_val) {
                cur_ptr = field_ptr;

                cur_type_id = fd.child_type_id;
            } else if (fd.field_type == .slice) {
                cur_ptr = field_ptr;

                cur_type_id = fd.child_type_id;
            }
        }
    }

```

```

        found = true;

        break;
    }

}

if (!found) return error.FieldNotFound;

seg_idx += 1;

}

return error.PathExhausted;

}

```

The walker processes path segments left to right. Each segment is either a field name or a numeric index.

For a field name: scan the current struct descriptor’s field array for a matching name. When found, add the field’s offset to the current pointer. If this is the last segment, return the pointer and descriptor. If not, check the field type — if it is a nested struct, set the current type to the child type index and continue. If it is a slice, do the same (the next segment will be a numeric index).

For a numeric index: read the slice’s data pointer from memory, multiply the index by the element size, advance the pointer. Continue with the next segment using the child type’s descriptor.

A path like `stats.values.0.value` resolves in four steps: “stats” adds an offset and follows `child_type_id` to the nested struct, “values” adds an offset and identifies a slice, “0” indexes into the slice by element size, “value” adds a final offset and returns the pointer.

7. Root Table

The walker needs a starting pointer. A hash map maps root names to base pointers and type indices:

```

const RootEntry = struct {
    ptr: [*]u8,

```

```

type_id: usize,

};

var roots: std.StringHashMap(RootEntry) = undefined;

pub fn registerRoot(name: []const u8, ptr: *anyopaque, type_id: usize) void {
    roots.put(name, .{
        .ptr = @ptrCast(ptr),
        .type_id = type_id,
    });
}

```

At startup:

```

registerRoot("player", @ptrCast(&player_entity), 0);
registerRoot("config", @ptrCast(&app_config), 3);
registerRoot("world", @ptrCast(&world_state), 7);

```

Path resolution splits on delimiter, looks up segment 0 in the root table, walks the remaining segments through the registry:

```

pub fn resolvePath(path: []const u8) !?WalkResult {
    var parts = split(path, '.');

    const root = roots.get(parts[0]) orelse return null;

    return walkPath(parts[1..], root.ptr, root.type_id);
}

```

For systems with arrays of records (entity pools, database tables), the root points to the array and the second segment is the numeric index. The existing walker handles numeric segments as slice indices, so `entity.5.stats.health` resolves naturally: “entity” looks up the root (pointing at the entity array), “5” indexes into the array, “stats” and “health” walk the struct fields.

8. Type Coercion

The walker returns a `WalkResult` with a raw pointer and a type tag. Type-specific getters and setters cast the pointer:

```
pub fn getInt(result: WalkResult) i32 {  
    const ptr: *i32 = @ptrCast(@alignCast(result.ptr));  
    return ptr.*;  
}  
  
pub fn setInt(result: WalkResult, value: i32) void {  
    const ptr: *i32 = @ptrCast(@alignCast(result.ptr));  
    ptr.* = value;  
}  
  
pub fn getFloat(result: WalkResult) f32 {  
    const ptr: *f32 = @ptrCast(@alignCast(result.ptr));  
    return ptr.*;  
}
```

These can be wrapped into a generic text interface that reads any field type as a string and writes any string to any field type:

```
pub fn getText(result: WalkResult) Text {  
    return switch (result.field.field_type) {  
        .int_i32 => Text.formatFrame("{} ", .{getInt(result)}),  
        .float_f32 => Text.formatFrame("{d:.2} ", .{getFloat(result)}),  
        .bool_val => if (getBool(result)) Text.init("true") else Text.init("fa  
        .enum_i32 => Text.formatFrame("{} ", .{getInt(result)}),  
        .text => getTextDirect(result),
```

```

        else => Text.init(""),
    };
}

pub fn setText(result: WalkResult, value: Text) bool {
    return switch (result.field.field_type) {
        .int_i32 => { setInt(result, value.toInt() orelse return false); return true; },
        .float_f32 => { setFloat(result, value.toFloat() orelse return false); return true; },
        .bool_val => { setBool(result, value.equalsRaw("true")); return true; },
        .enum_i32 => { setInt(result, value.toInt() orelse return false); return true; },
        .text => { setTextDirect(result, value); return true; },
        else => false,
    };
}

```

Property editors and data binding systems call `getText` and `setText` with a path string. They do not need to know the field type in advance. The descriptor's type tag handles the conversion at the point of access.

9. What This Replaces

A code generation pipeline for 140 structs typically produces per struct: a JSON serializer function, a JSON deserializer function, a data binding registration function, a property editor builder function, and bridge accessor functions. Five generated functions per struct, 700 generated functions total, thousands of lines of generated source entering the compiler every build.

The registry replaces all generated code with:

- One `buildRegistry` call producing a static array (build time, zero runtime cost)
- One generic serializer that walks field descriptors and writes by type tag
- One generic deserializer that walks field descriptors and reads by type tag

- One generic property panel renderer that walks field descriptors and creates inputs by type tag
- One generic path walker that resolves any path to any field at any depth

Five generic functions replace 700 generated functions. Adding struct 141 means adding one line to the type tuple. No generator to run, no output to synchronize, no generated source to compile.

In the reference project, removing the code generation pipeline reduced compile time from 15 seconds to approximately 2 seconds — the time to compile the actual application code without thousands of generated lines.

10. Implementation in Other Languages

The technique requires two capabilities: determining field byte offsets at build time, and performing pointer arithmetic at runtime. Every compiled language has both.

10.1 C

C provides `offsetof()` in `<stddef.h>` and `sizeof()` as a language primitive. Write a separate program that includes the application's headers:

```
#include "entity.h"

#include <stddef.h>

#include <stdio.h>

void describe_Entity(StructDescriptor* sd) {
    strcpy(sd->name, "Entity");

    sd->struct_size = sizeof(Entity);

    sd->field_count = 0;

    int i = sd->field_count++;

    strcpy(sd->fields[i].name, "health");

    sd->fields[i].offset = offsetof(Entity, health);

    sd->fields[i].size = sizeof(int);
}
```

```

sd->fields[i].field_type = TYPE_INT32;

}

int main() {
    StructDescriptor descriptors[128];

    int count = 0;

    describe_Entity(&descriptors[count++]);

    // ... all structs

    FILE* f = fopen("struct_registry.bin", "wb");

    fwrite(&count, sizeof(int), 1, f);

    fwrite(descriptors, sizeof(StructDescriptor), count, f);

    fclose(f);

}

```

Compile and run this tool before building the main application. The output is a binary file loaded at startup. The main application walks structs using the same pointer arithmetic as the Zig implementation. The per-struct `describe_` functions are manual but mechanical — a simple script can generate them from header files. Critically, this script produces a data file, not source files that enter compilation.

10.2 C++

Same `offsetof()` approach as C. A macro reduces per-field boilerplate:

```

#define FIELD(Struct, field, type) \
    addField(sd, #field, offsetof(Struct, field), \
             sizeof(((Struct*)0)->field), type)

void describe_Entity(StructDescriptor& sd) {
    strncpy(sd.name, "Entity", 63);

    sd.struct_size = sizeof(Entity);
}

```



```
FIELD(Entity, health, FieldType::Int32);

FIELD(Entity, speed, FieldType::Float32);

FIELD(Entity, name, FieldType::Text);

}
```

Alternatively, use libclang as a build step to parse headers and extract all struct layouts automatically from the AST. Zero manual per-struct code.

10.3 Rust

Proc macros run at compile time and can inspect struct definitions:

```
#[derive(Described)]

struct Entity {

    health: i32,

    speed: f32,

    name: Text,

}
```

The `#[derive(Described)]` macro generates a `const DESCRIPTOR: StructDescriptor` for each struct using `offset_of!` and `std::mem::size_of`. Written once, applied to every struct with one attribute. The compiler expands it at build time. The result is static const data embedded in the binary.

10.4 Go

Use `go generate` with a build tool that parses source files and computes layouts using `unsafe.Offsetof`. Output a JSON descriptor file loaded at startup. Runtime field access uses `unsafe.Pointer` arithmetic with the pre-computed offsets.

10.5 Common Pattern

Regardless of language, the implementation follows three steps:

1. At build time, determine each field's name, byte offset, byte size, and type classification. Use the language's compile-time facilities or a separate build tool.
2. Store the metadata as a static data structure embedded in the binary or as an external file loaded at startup.

3. At runtime, resolve paths by iterating descriptors, adding offsets to base pointers, and interpreting bytes according to type tags.
-

11. Minimal Implementation Checklist

What is required to use this technique:

- Existing structs. No modifications, no annotations, no base classes, no interface implementations.
- The registry module. `FieldDescriptor`, `StructDescriptor`, `WalkResult`, `FieldType` enum, `classifyFieldType`, `buildRegistry`, `walkPath`. Approximately 150 lines.
- A type list declaring which structs to register. One line per struct.
- A root table mapping names to base pointers. One call per root object at startup.
- Getter/setter functions per type needed. Each is 3-4 lines: resolve path, cast pointer, read or write.

What is not required:

- No reflection library
 - No code generation tool or build script
 - No annotation or attribute system
 - No framework or dependency
 - No base class inheritance
 - No per-field registration calls
 - No schema definition file separate from the struct source
-

12. Limitations

The type list must be maintained manually. Adding a struct to the system requires adding one line to the tuple. If a struct is omitted, its fields are not navigable. The compiler does not warn about omissions.

Field name lookups are linear scans of the field array, up to 96 entries per nesting level. In practice this is fast — 96 short-string comparisons is negligible relative to any I/O or rendering work — but it is $O(N)$ per segment, not $O(1)$.

The fixed 64-byte name buffer truncates field names longer than 64 characters. No struct in the reference project approached this limit, but generated or macro-expanded names in other projects might.

The fixed 96-field maximum per struct prevents registration of very large flat structs. Structs exceeding this limit must be split or the constant increased.

Enum values are stored as their backing integer. The enum variant name is not preserved in the descriptor. A field with type `ContentItemType.Tileset` reads as integer 14, not as the string "Tileset". Enum name resolution requires a separate lookup table.

Slice walking requires the element type to be registered in the type tuple. A slice of a struct type not in the registry will have `child_type_id = 0`, pointing at whatever struct is first in the registry. The build-time function does not error on this — it silently returns index 0. Adding a compile-time check for unregistered element types would improve safety.

The technique provides no access control. Any code with a base pointer and the registry can read or write any field. Access restrictions, if needed, must be implemented in the path resolution layer above the walker.

References

No external references. The technique was developed independently to solve a specific compile-time inflation problem in a 140+ struct Zig project. The `offsetof` primitive used in C implementations has existed since C89 (1989). Zig's `@offsetof` and `@sizeof` builtins provide equivalent functionality at comptime.

Appendix A: Field Type Classification Coverage

Field	Zig Type	Size (bytes)	Requires child type id	Requires slice element size	Runtime Cast Target
int i32	i32	4	No	No	*i32
float f32	f32	4	No	No	*f32
bool val	bool	1	No	No	*bool
uint u8	u8	1	No	No	*u8
uint u16	u16	2	No	No	*u16
int usize	usize	8 (64-bit)	No	No	*usize
text	Text	4096+	No	No	*Text
string	String	heap-backed	No	No	*String
enum i32	any enum(i32)	4	No	No	*i32

FieldType	Zig Type	Size (bytes)	Requires child type id	Requires slice element size	Runtime Cast Target
struct	any	varies	Yes	No	*anyopaque,
val	struct				recurse
slice	[]T	16 (ptr+len)	Yes (if element is struct)	Yes	read ptr, index by element size
vector2	struct{x:f32,y:f32}	32	No	No	*Vector2
unknown	anything else	varies	No	No	not accessible

Appendix B: Detection Logic for Ambiguous Types

Some types cannot be classified by primitive identity alone. The classifier applies these checks in order. First match wins.

Priority	Check	Result	Rationale
1	T == i32	.int i32	Exact identity, no ambiguity
2	T == f32	.float f32	Exact identity
3	T == bool	.bool val	Exact identity
4	T == u8	.uint u8	Exact identity
5	T == u16	.uint u16	Exact identity
6	T == usize	.int usize	Exact identity
7	[?](T) == .@“enum”	.enum i32	All enums treated as i32 regardless of backing type
8	T == Text	.text	Identity check before structural, prevents Text matching as struct val
9	T == String	.string	Identity check before structural
10	has x:f32, has y:f32, no z	.vector2	Structural signature match, distinguishes Vector2 from Vector3
11	[?](T) == .@“struct”	.struct val	Catch-all for remaining structs
12	pointer with slice size	.slice	Detects []T patterns
13	none of above	.unknown	Field will not be navigable

Adding a new custom type (e.g., Color, Rect, Matrix4) requires inserting an identity check between priorities 9 and 10 in the classifier function.

Appendix C: Path Segment Resolution Rules

Each segment of a dot-delimited path is resolved by exactly one rule. The walker applies these in order per segment.

Segment Content	Current Type	Action	Pointer Update	Type Update
Numeric string	slice (after slice field)	Index into slice data	$\text{ptr} = \text{slice data ptr} + (\text{index} \times \text{element size})$	type id = child type id of the slice field
Field name (last segment)	struct val	Return WalkResult	$\text{ptr} = \text{cur ptr} + \text{field.offset}$	none, return field descriptor
Field name → struct val	struct val	Descend into nested struct	$\text{ptr} = \text{cur ptr} + \text{field.offset}$	type id = field.child type id
Field name → slice	struct val	Prepare for index segment	$\text{ptr} = \text{cur ptr} + \text{field.offset}$	type id = field.child type id
Field name → primitive (not last)	struct val	Error: cannot descend into primitive	none	error.FieldNotFound
Unrecognized name	any	Error: field does not exist	none	error.FieldNotFound

Appendix D: Memory Layout of the Registry Itself

For a project with N registered struct types, each with up to 96 fields:

Component	Size Per Entry	Count	Total
StructDescriptor.name	64 bytes	N	64N bytes
StructDescriptor.name len	8 bytes	N	8N bytes

Component	Size Per Entry	Count	Total
StructDescriptor.struct size	8 bytes	N	8N bytes
StructDescriptor.field count	8 bytes	N	8N bytes
FieldDescriptor (per struct)	96 × 88 bytes	N	8,448N bytes
Total per struct type	8,536 bytes		
Project Scale	Struct Count	Registry Size	Percentage of 50MB Binary
Small	20	167 KB	0.33%
Medium	60	500 KB	0.98%
Large (reference project)	140	1.17 MB	2.28%
Very large	300	2.50 MB	4.88%

The registry is static data. It occupies space in the binary but requires no runtime allocation, no initialization beyond loading the binary into memory, and no deallocation.

Appendix E: Walk Performance Characteristics

Path Depth	Example Path	Name Lookups	Pointer Additions	Slice Indexing Ops
1	health	1	1	0
2	stats.health	2	2	0
3	stats.values.0	2	2	1
4	stats.values.0.value3	3	3	1
6	skills.0.levels.0.success.base modifier	4	4	2

Each name lookup scans up to `field_count` entries (worst case 96, typical 10-30). Each scan compares bytes in the fixed name buffer. Each pointer addition is a single integer add. Each slice index is a multiply and add.

Operation	Approximate Cost	Notes
Name lookup (10 fields)	~5 ns	Linear scan, short strings, cache-hot
Name lookup (30 fields)	~15 ns	Still cache-hot, single cache line per descriptor
Name lookup (96 fields)	~50 ns	Worst case, rare in practice
Pointer addition	~1 ns	Single integer add
Slice index	~2 ns	Multiply + add
Full 4-segment path	~20-40 ns	Typical case
Full 6-segment path	~30-60 ns	Deep nested with slices

For comparison, a hash map lookup for a single DataRouter registration is typically 50-200 ns depending on hash function and table load. The registry walker at 4 segments is competitive with or faster than a single hash lookup, while resolving to arbitrary depth.

Appendix F: Failure Modes and Behavior

Failure	Cause	Walker Behavior	Caller Sees
Field name not found	Typo in path, field renamed, struct changed	Returns <code>error.FieldNotFound</code>	Error return, caller decides handling
Path too short	Stopped at struct val, not a leaf field	Returns <code>error.PathExhausted</code>	Error return
Slice index out of bounds	Index exceeds slice length	Pointer arithmetic proceeds, reads garbage	Undefined behavior — caller must validate index
Struct type not in registry	Omitted from type tuple	child type id defaults to 0, walks wrong struct	Silent incorrect results
Field count exceeds 96	Struct has more than 96 fields	Fields beyond 96 not recorded	Those fields invisible to walker

Failure	Cause	Walker Behavior	Caller Sees
Name exceeds 64 bytes	Very long field name	Truncated, may not match path segment	<code>error.FieldNotFound</code> on that field
Enum variant interpretation	Enum stored as i32	Reads/writes integer value	Caller receives integer, not variant name

Two of these failures are silent (out-of-bounds slice index, unregistered child type). Both can be addressed with optional validation: bounds-checking the slice index against the slice length read from memory, and emitting a compile-time error when `findTypeIndex` returns 0 for a non-first-entry type.

Appendix G: Comparison to Alternative Approaches

Characteristic	Code Generation	Manual Registration	Runtime Reflection (Java/C#)	Comptime Registry
Build-time cost	Generator runs + compiles output	None	None	Comptime evaluation
Compile-time impact	Linear with struct count (thousands of generated lines)	None	None	Constant (one function, output is static data)
Runtime cost	Zero (compiled to direct access)	Hash lookup per access	Dynamic dispatch + allocation	Pointer arithmetic + name scan
Maintenance burden	Generator tool + registry file + sync discipline	Per-field registration calls	None (language provides)	One line per struct in type tuple
Can drift from source	Yes (generator not re-run)	Yes (registration not updated)	No (reads live type info)	No (reads struct definitions at comptime)
Heap allocation	None at runtime	Registration table at startup	Per-query in many implementations	None

Characteristic	Code Generation	Manual Registration	Runtime Reflection (Java/C#)	Comptime Registry
Works in C	Yes	Yes	No	Via build tool (not comptime)
Works in C++	Yes	Yes	Limited (RTTI)	Via build tool or libclang
Works in Rust	Yes	Yes	No	Via proc macro
Works in Zig	Yes	Yes	No	Native comptime
Works in Go	Yes	Yes	Yes (reflect pkg, slow)	Via go generate
Works in Java/C#	Unnecessary (has reflection)	Unnecessary	Native	Unnecessary
Arbitrary depth navigation	If generated for each path	If registered for each path	Yes	Yes
Handles struct addition	Re-run generator, recompile output	Write new registration calls	Automatic	Add one line to type tuple
Handles field addition	Re-run generator, recompile output	Write new registration call	Automatic	Automatic (comptime reads new field)

Appendix H: Compile-Time Cost Analysis

The `buildRegistry` function runs at compile time. Its cost depends on the number of types and fields.

Types	Avg Fields/Type	Total Fields Processed	Eval Branch Quota Needed	Measured Zig Comptime Duration
20	15	300	~50,000	<100 ms
60	20	1,200	~200,000	~200 ms
140	25	3,500	~600,000	~500 ms
300	30	9,000	~1,000,000	~1,200 ms

The `@setEvalBranchQuota(1000000)` directive is required for large reg-

istries. Without it, the compiler aborts after its default branch quota (typically 1,000) is exceeded during the inline for loops.

Comptime evaluation cost is paid once per compilation. The generated static data requires zero initialization at program startup. Contrast with manual registration, which executes registration calls at startup proportional to total field count, or runtime reflection, which builds type metadata lazily on first access with allocation.

Appendix I: Extension Points

The registry as presented is minimal. These extensions do not require changes to the core data structures or walker.

Extension	Implementation	Modifies Core	Use Case
Field attributes (readonly, hidden, min/max)	Additional fields on FieldDescriptor or separate attribute table keyed by type id + field index	FieldDescriptor grows	Property editors that enforce constraints
Enum variant name resolution	Separate table mapping enum type + i32 value → string	No	Display enum names in editors instead of integers
Cached path resolution	Hash map from path string → (type id, offset chain)	No	Repeated access to same path avoids re-walking
Change notification	Wrapper around setters that calls a callback after write	No	Dirty marking, UI refresh, persistence triggers
Diff between two instances	Walk both instances with same descriptor, compare per field	No	Undo/redo, save delta, network sync
Clone/deep copy	Walk source instance, copy bytes per field, deep-copy slices	No	Entity template instantiation
JSON serialization	Walk fields, write name:value pairs, recurse for struct val, iterate for slice	No	Persistence, import/export
Binary serialization	Walk fields, write raw bytes in offset order	No	Network packets, fast save/load

Extension	Implementation	Modifies Core	Use Case
Schema comparison	Compare two registries (different binary versions) field by field	No	Migration detection, compatibility checking

Appendix J: Language Feature Requirements

Minimum language capabilities needed to implement each variant of the technique.

Capability	Required For	C	C++	Rust	Zig	Go
offsetof or equivalent	Capturing field offsets	offsetof macro	offsetof() macro	offsetof! macro	[?] builtin	unsafe.Offsetof
sizeof or equivalent	Capturing field and struct sizes	sizeof operator	sizeof operator	std::mem::size of	[?] builtin	unsafe.Sizeof
Compile-time code execution	Building registry without build tool	No	Limited (const-expr)	Proc macros	Full comp-time	No
Pointer arithmetic	Runtime field access	Native	Native	unsafe block	[?] + arithmetic	unsafe.Pointer
Type introspection at build time	Automatic field enumeration	No	No (without lib-clang)	Proc macro syn crate	[?] builtin	No (without AST parsing)
Static data embedding	Storing registry in binary	const arrays	constexpr arrays	const statics	comptime const	var at package level

Languages without compile-time code execution (C, Go) require a separate build tool that compiles and runs before the main application. The build tool includes the same headers or parses the same source files, extracts layout information, and writes a data file. The main application loads this file at startup. The technique is identical; only the extraction mechanism differs.

[[HOWL-COMP-13-2026](#)] Runtime Struct Navigation in Compiled Languages Without

Reflection or Code Generation. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-13-2026>

[[HOWL-COMP-1-2026](#)] Implementing a Tall-Infra Data-Only Execution System .
Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-1-2026>

[[HOWL-COMP-2-2026](#)] Tall-Infra, Data-Only Development. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-2-2026>

[[HOWL-COMP-3-2026](#)] Silo OS. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-3-2026>

[[HOWL-COMP-4-2026](#)] Geometric Security. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-4-2026>

[[HOWL-COMP-5-2026](#)] Partial Geometric Security. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-5-2026>

[[HOWL-COMP-6-2026](#)] SIML - Silo Markup Language (TOML Format). Github:
<https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-6-2026>

[[HOWL-COMP-7-2026](#)] SiQL. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-7-2026>

[[HOWL-COMP-8-2026](#)] SiloSec Protocol Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-8-2026>

[[HOWL-COMP-9-2026](#)] Building Applications with OpsDB Application Architecture.
Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-9-2026>

[[HOWL-COMP-10-2026](#)] Tall-Infra Data-Only Execution. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-10-2026>

[[HOWL-COMP-11-2026](#)] Name Driven Development. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-11-2026>

[[HOWL-COMP-12-2026](#)] Closed Loop Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-12-2026>